Beginner | Neural network fundamentals | Highlights

# Inside a Neuron: The Building Blocks of a Neural Network & AI

Compute a single artificial neuron's output by hand and understand what weights, bias, and activation functions actually do.

For people who have heard the term 'neural network' and want to understand, in concrete arithmetic, what a single neuron computes · 27 July 2026

The one- or two-page version: what matters, the topics it covers, and every term defined. Read the full lesson for the explanations and worked examples.

Watch the source video | Read the full lesson | Read the condensed version

## What matters

- ✓ A neuron turns an input vector into a single output signal by computing a weighted sum of the inputs, adding a bias, then passing the result through an activation function.
- ✓ Weights encode how much a specific neuron cares about each input feature; every neuron in a layer sees the same input vector but specializes by learning a different set of weights.
- ✓ The bias shifts a neuron's activation threshold, letting it fire even when the weighted sum of its inputs is small or zero.
- ✓ Activation functions like sigmoid or ReLU introduce nonlinearity. Without them, stacking any number of neurons would still compute nothing more than one overall linear function.
- ✓ A neuron's activation level (its output after the activation function) is a measure of how strongly the pattern that neuron detects is present in the current input.
- ✓ Training a network means repeatedly adjusting every neuron's weights and biases to reduce prediction error; backpropagation is the algorithm that works out how to adjust them, though it isn't covered in depth here.
- ✓ Tools like MLflow don't change what a neuron computes — they log and visualize how weights and biases evolve across training runs so engineers can compare configurations.

## What it covers

The raw weighted sum can be any number, so a neuron adds a bias and then applies an activation function like sigmoid to squash the result into a bounded, usable range.

06 **Why Nonlinearity Lets Networks Learn Complex Patterns** `3:35`

The activation function is what makes a network nonlinear; without it, stacking any number of neurons would collapse into a single linear model.

07 **From One Neuron to a Network: Stacking, Weights, and Training** `4:20`

A single neuron can only capture one relationship, but stacking many, fully connected between layers, lets a network learn combinations of patterns; training is the process of tuning every weight and bias.

## Glossary

**Neuron** — The basic unit of a neural network. It takes a vector of numbers as input and produces one output number by computing a weighted sum, adding a bias, and applying an activation function.

**Vector** — An ordered list of numbers, where each position represents a specific measurable feature of the data being described.

**Weight** — A learned number attached to one input feature of one neuron, representing how much that feature matters to that neuron's output.

**Weighted sum** — The result of multiplying each input by its matching weight and adding all the products together; mathematically a dot product of the input vector and the weight vector.

**Bias** — A single learned number added to a neuron's weighted sum before activation. It shifts the neuron's activation threshold independent of any input feature.

**Activation function** — A function applied to a neuron's weighted sum (plus bias) that squashes it into a bounded, usable output and introduces nonlinearity into the network.

**Sigmoid function** — An activation function that compresses any real number into the range between 0 and 1, commonly used to conceptualize a neuron's output as a confidence level.

**ReLU** — Rectified Linear Unit, an activation function that outputs the input directly if it's positive and 0 otherwise. Used more often than sigmoid in modern networks for its computational efficiency.

**Activation level** — A neuron's output after its activation function has been applied; a higher activation level means the pattern that neuron detects is more strongly present in the current input.

**Nonlinearity** — A property of a function whose output isn't a fixed straight-line multiple of its input. Activation functions must be nonlinear for stacking multiple layers of neurons to add any representational power.

**Layer** — A group of neurons that all receive the same input vector, each computing its own weighted sum, bias, and activation independently.

**Backpropagation** — The training algorithm that works backward from a network's prediction error to determine how much each weight and bias contributed to that error, so they can be adjusted to reduce it.